

bok25

基
礎
班

、考研数据結構



平衡二叉树

AVL



平衡二叉树 定义

AVL

平衡二叉树或者是空树，或者是具有以下特征的二叉排序树

- 左子树和右子树的深度之差的绝对值不超过1
- 左子树和右子树也是平衡二叉树



平衡二叉树 定义

AVL

平衡二叉树或者是空树，或者是具有以下特征的二叉排序树

- 左子树和右子树的深度之差的绝对值不超过1
- 左子树和右子树也是平衡二叉树

一棵平衡二叉树必定是一棵二叉排序树

AVL上的节点的左子树和右子树的深度之差称为平衡因子。

它只可能是-1、0、1。

只有一棵BST所有节点的平衡因子的绝对值都小于1，这棵树才是AVL



平衡二叉树性质

AVL

假设以 n_h 表示深度为 h 的平衡二叉树中含有的最少节点数，AVL具有以下递推公式：

$$n_h = n_{h-2} + n_{h-1} + 1$$

显然 $n_0=0, n_1=1, n_2=2$ ，由此可以推出后序所有 n_h



平衡二叉树

AVL

查找



平衡二叉树查找

AVL

AVL是BST的基础上增加限制，故查找操作和BST相同

- 若平衡二叉树为空，则查找失败，返回空指针
- 若平衡二叉树非空，将给定值key与根节点的关键字 $T \rightarrow \text{data.key}$ 进行比较
 - 若key等于 $T \rightarrow \text{data.key}$ ，则查找成功，返回根节点地址
 - 若key小于 $T \rightarrow \text{data.key}$ ，则递归查找左子树
 - 若key大于 $T \rightarrow \text{data.key}$ ，则递归查找右子树

时间复杂度为 $O(\log_2 n)$

平衡二叉树

AVL

插入操作



平衡二叉树插入操作

AVL

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

插入后导致AVL不平衡，则调整范围就是**最小不平衡子树**



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

左旋：以某节点为旋转节点，其右子节点变为旋转节点的父节点，右子节点的左子节点变为旋转节点的右子节点



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

左旋：以某节点为旋转节点，其右子节点变为旋转节点的父节点，右子节点的左子节点变为旋转节点的右子节点

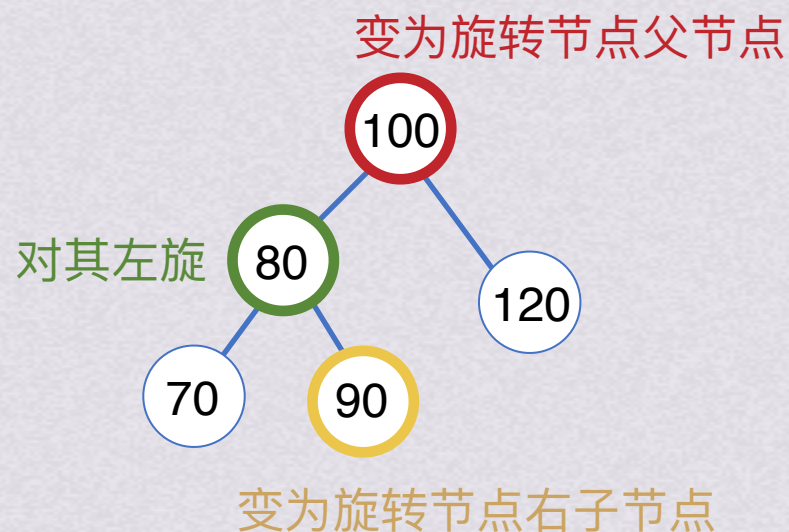




平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

左旋：以某节点为旋转节点，其右子节点变为旋转节点的父节点，右子节点的左子节点变为旋转节点的右子节点

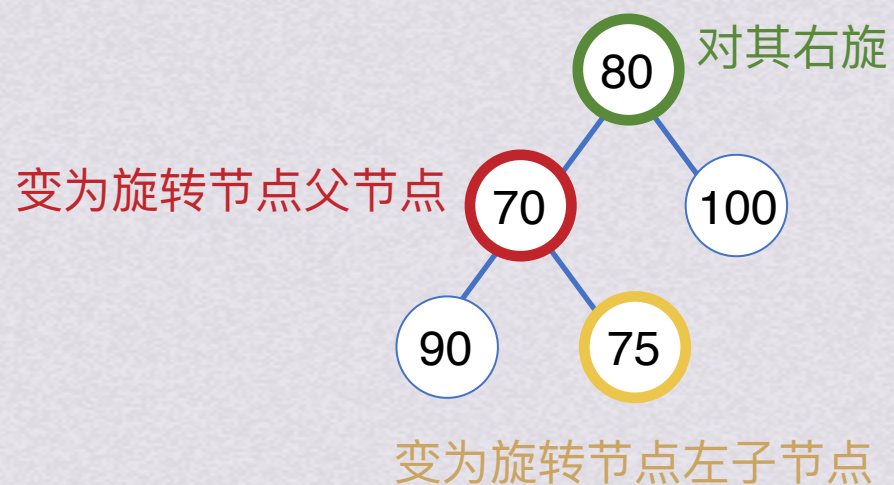




平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

右旋：以某节点为旋转节点，其左子节点变为旋转节点的父节点，左子节点的右子节点变为旋转节点的左子节点

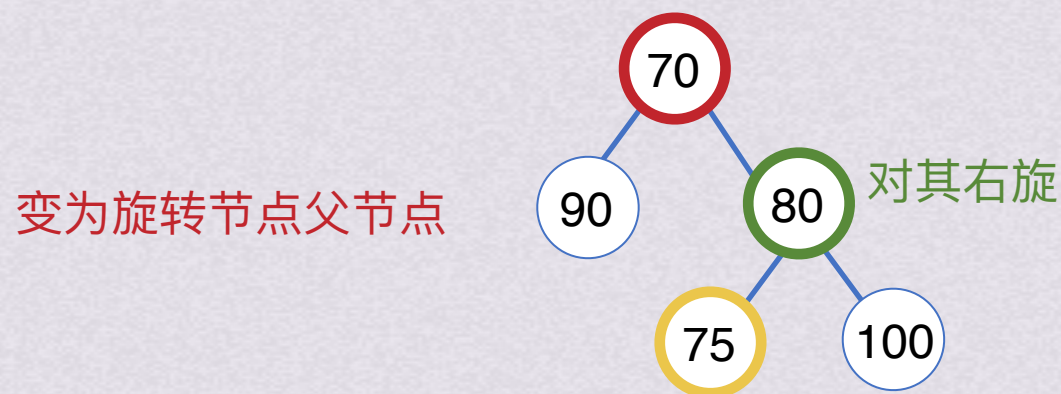




平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

右旋：以某节点为旋转节点，其左子节点变为旋转节点的父节点，左子节点的右子节点变为旋转节点的左子节点



变为旋转节点左子节点

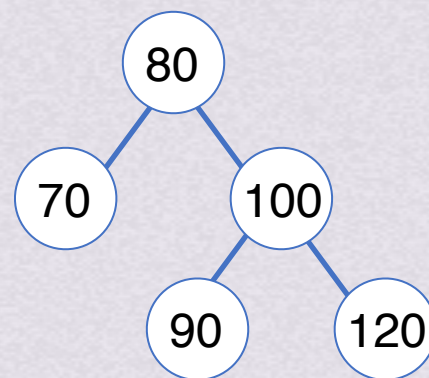


平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡

执行插入68





平衡二叉树插入操作

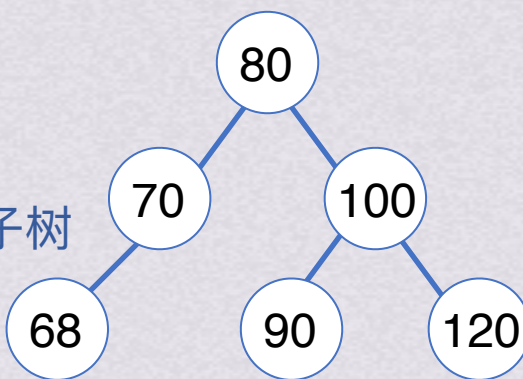
最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡

$68 < 80$
对比左子树

执行插入68

$68 < 70$
成为其左子树





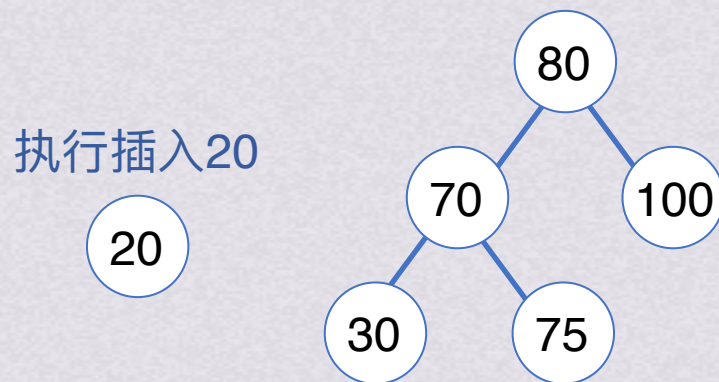
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点





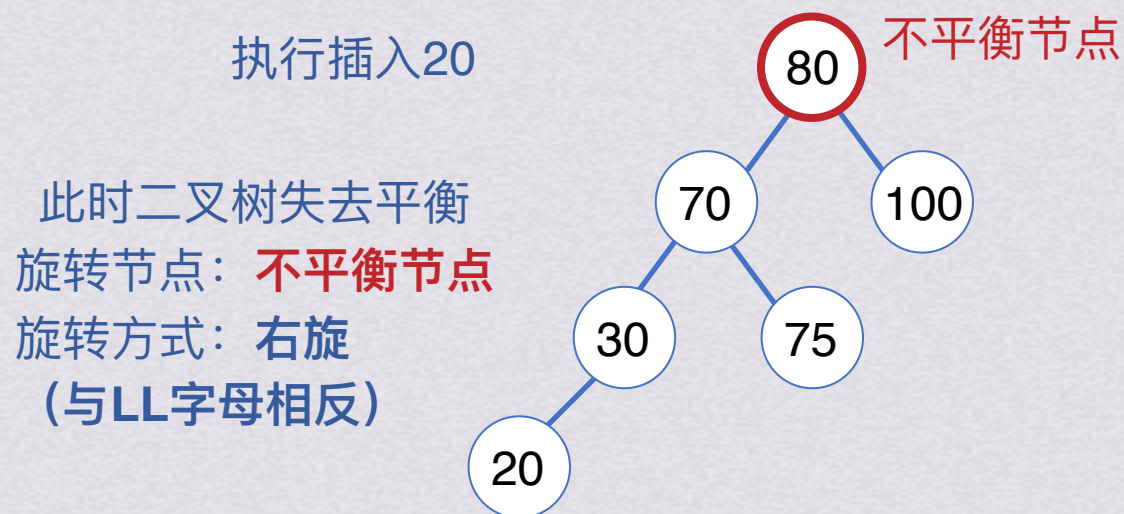
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点





平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

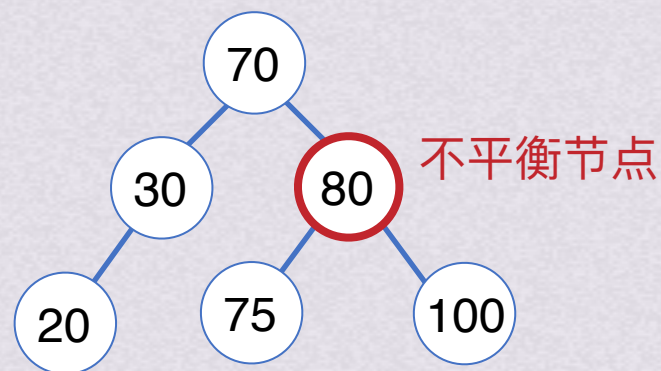
情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点

执行插入20

此时二叉树失去平衡
旋转节点：**不平衡节点**
旋转方式：**右旋**
(与LL字母相反)



树重新达到平衡



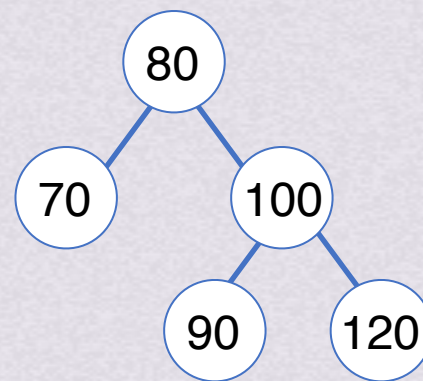
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点



执行插入140





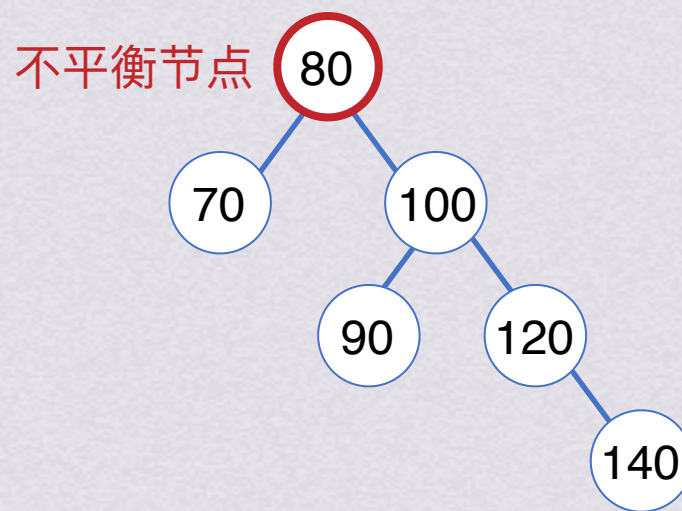
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点



执行插入140

此时二叉树失去平衡
旋转节点：**不平衡节点**
旋转方式：左旋
(与RR字母相反)



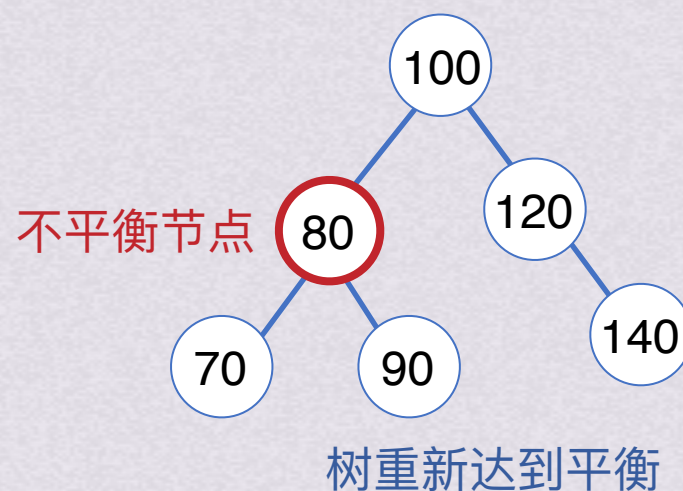
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点



执行插入140

此时二叉树失去平衡
旋转节点：**不平衡节点**
旋转方式：**左旋**
(与RR字母相反)



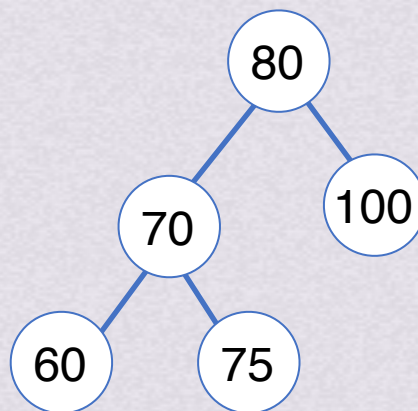
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

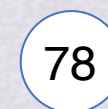
情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点



执行插入78





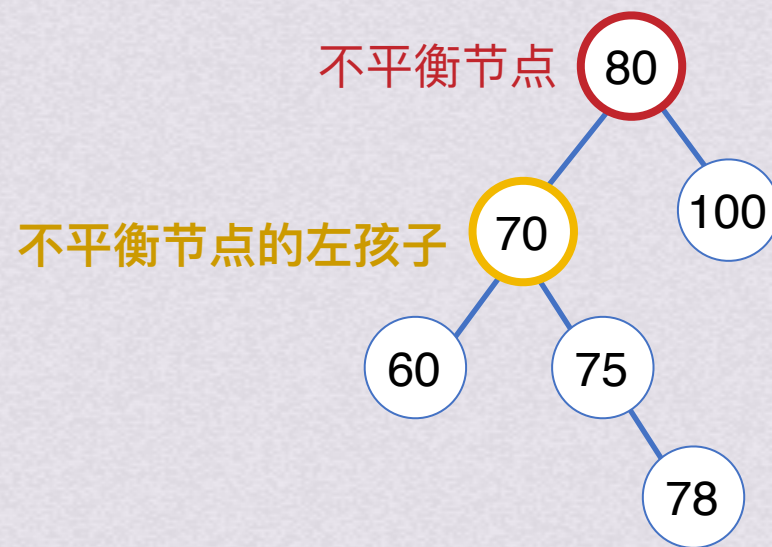
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点



此时二叉树失去平衡

旋转节点：不平衡节点的左孩子

旋转方式：左旋

(与LR字母一致)



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点



此时二叉树失去平衡

旋转节点：不平衡节点的左孩子

旋转方式：左旋

(与LR字母一致)



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点



此时变为LL型

旋转节点：**不平衡节点**

旋转方式：**右旋**

(与LL字母相反)

此时二叉树失去平衡

旋转节点：**不平衡节点的左孩子**

旋转方式：**左旋**

(与LR字母一致)



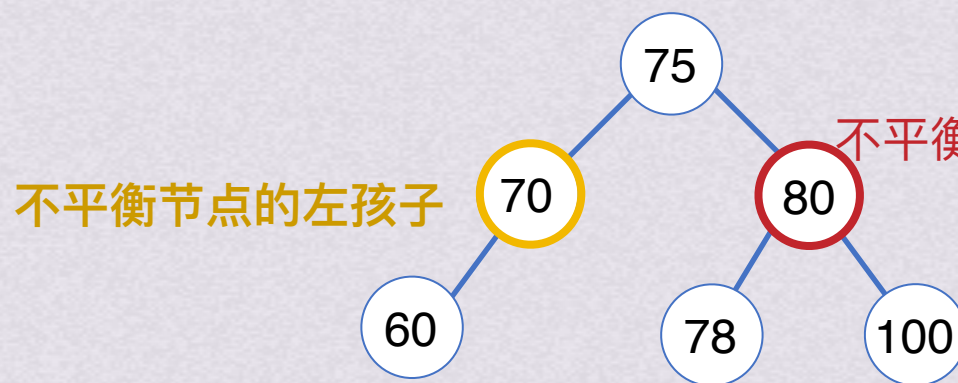
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点



此时树重新达到平衡

此时变为LL型

旋转节点：不平衡节点

旋转方式：右旋

(与LL字母相反)

此时二叉树失去平衡

旋转节点：不平衡节点的左孩子

旋转方式：左旋

(与LR字母一致)



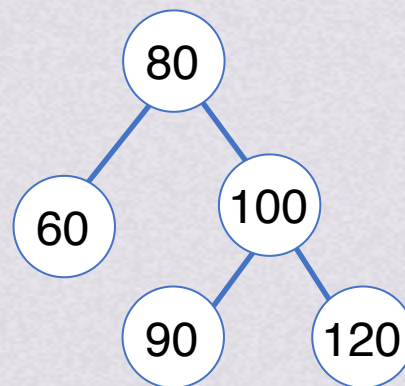
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点
- **RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点。**



执行插入95





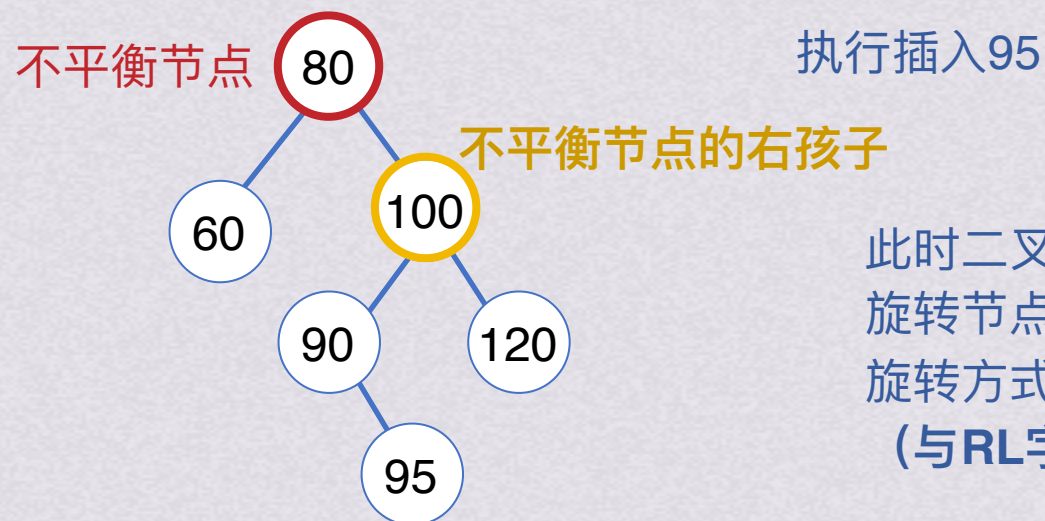
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点
- **RL**：在最小不平衡子树根节点右子树根节点的左子树上插入节点。



此时二叉树失去平衡

旋转节点：**不平衡节点的右孩子**

旋转方式：**右旋**

(与RL字母一致)



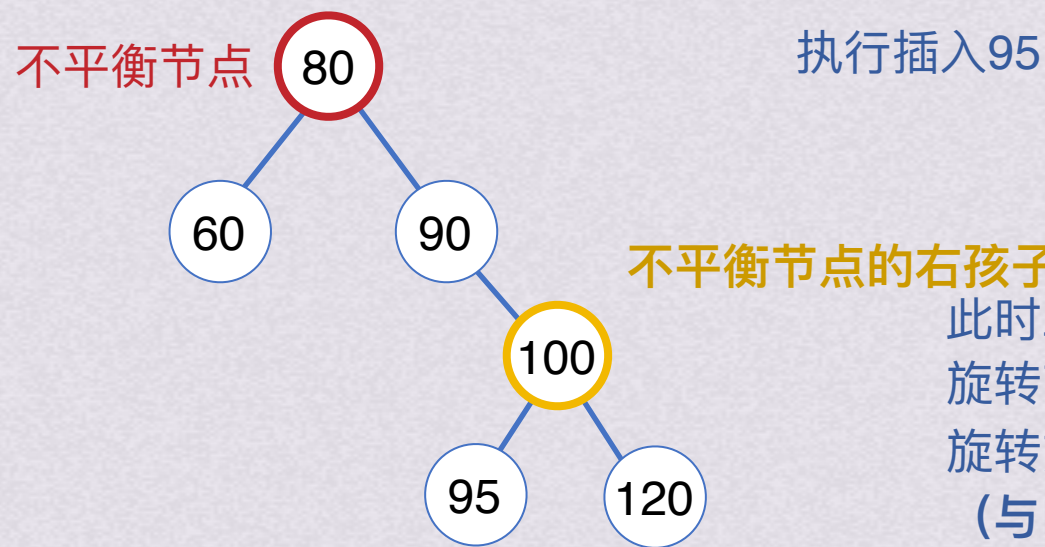
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点
- **RL**：在最小不平衡子树根节点右子树根节点的左子树上插入节点。



此时二叉树失去平衡
旋转节点：**不平衡节点的右孩子**
旋转方式：右旋
(与RL字母一致)



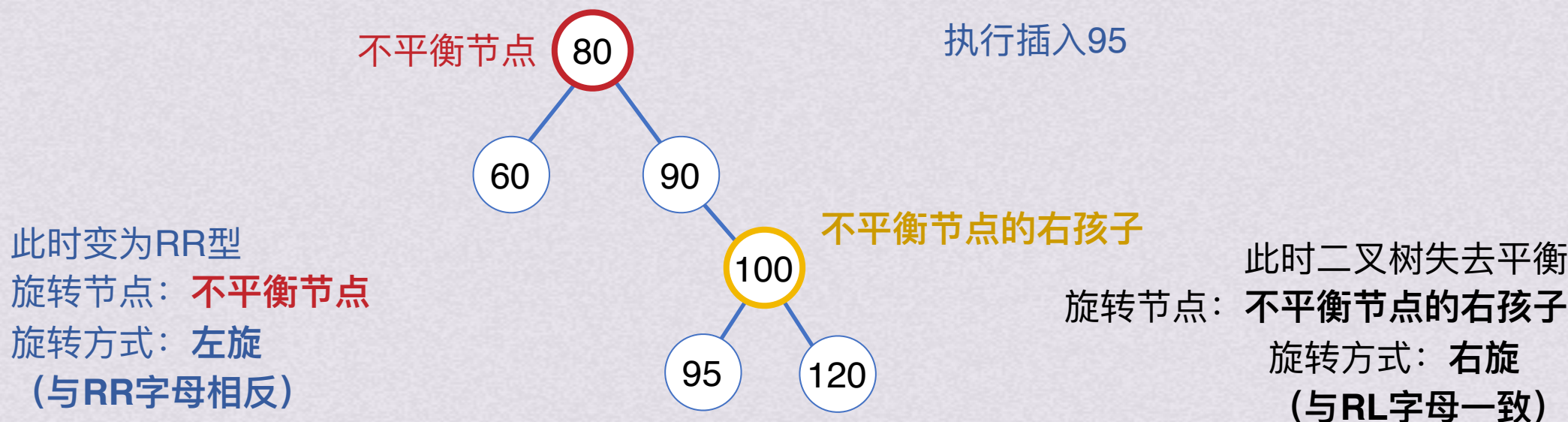
平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点
- **RL**：在最小不平衡子树根节点右子树根节点的左子树上插入节点。





平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点
- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点
- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点
- **RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点。**

执行插入95

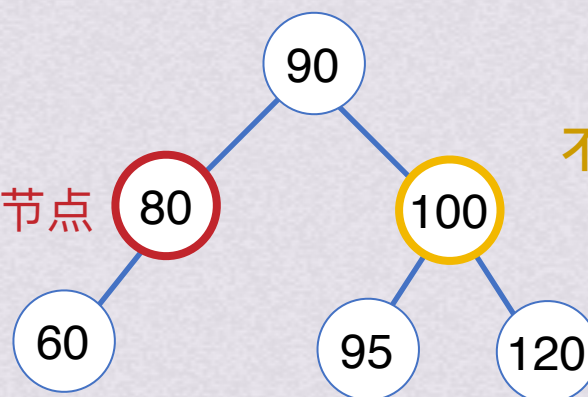
重新达到平衡

此时变为RR型

旋转节点：**不平衡节点**

旋转方式：**左旋**
(与RR字母相反)

不平衡节点



不平衡节点的右孩子

此时二叉树失去平衡
旋转节点：**不平衡节点的右孩子**
旋转方式：**右旋**
(与RL字母一致)



平衡二叉树插入操作

最小不平衡子树：离插入节点最近且平衡因子绝对值超过1的祖先节点，以该节点为根的子树称为最小不平衡子树

情况一：插入节点后仍然平衡，不做任何调整

情况二：插入节点后不平衡

- LL：在最小不平衡子树根节点左子树根节点的左子树上插入节点

右旋不平衡节点

- RR：在最小不平衡子树根节点右子树根节点的右子树上插入节点

左旋不平衡节点

- LR：在最小不平衡子树根节点左子树根节点的右子树上插入节点

先左旋不平衡节点左孩子 再右旋不平衡节点

- RL：在最小不平衡子树根节点右子树根节点的左子树上插入节点。

先右旋不平衡节点右孩子 再左旋不平衡节点



平衡二叉树

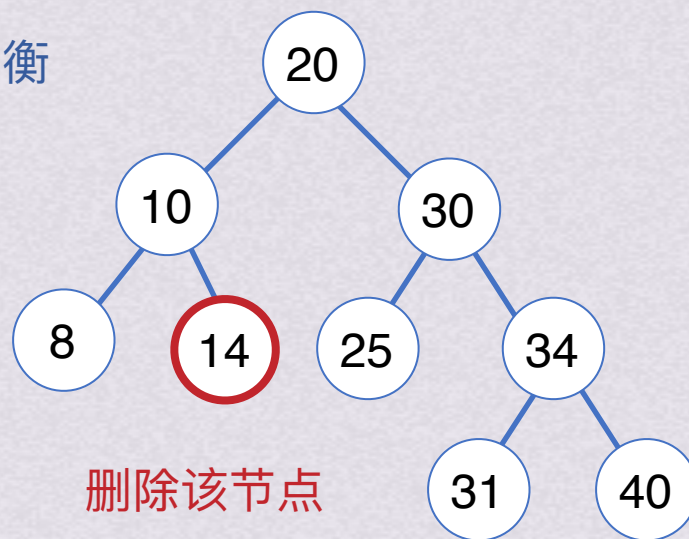
删除操作



平衡二叉树删除操作

删除叶子节点情况

情况一：删除后不影响平衡



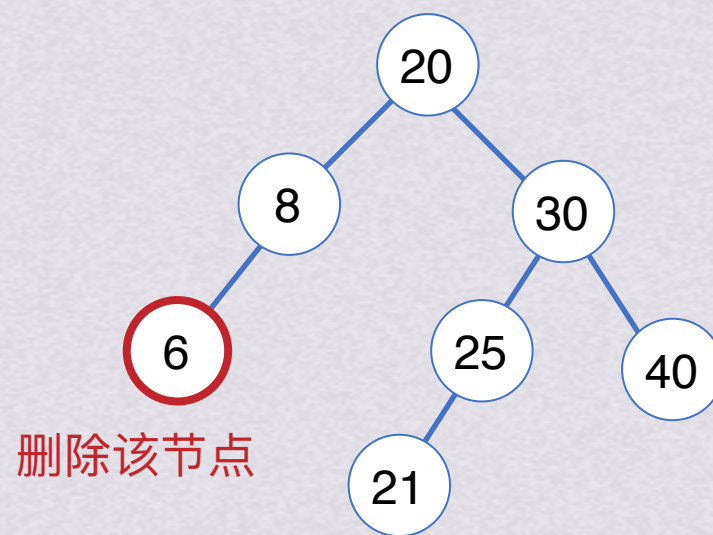


平衡二叉树删除操作

删除叶子节点情况

情况一：删除后不影响平衡，直接删除

情况二：删除后不再是平衡二叉树



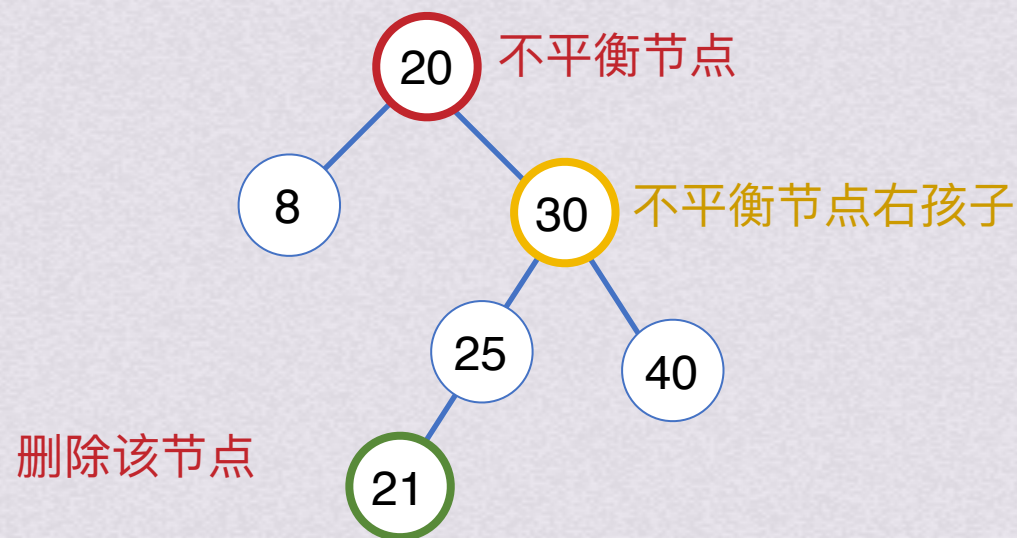


平衡二叉树删除操作

删除叶子节点情况

情况一：删除后不影响平衡，直接删除

情况二：删除后不再是平衡二叉树



等价于21节点为RL型插入

先右旋后左旋解决

先右旋不平衡节点右孩子 再左旋不平衡节点

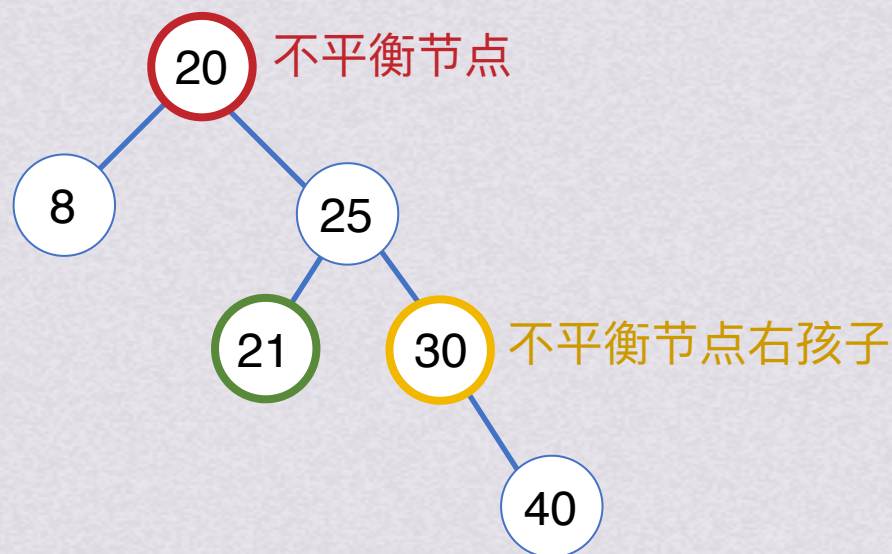


平衡二叉树删除操作

删除叶子节点情况

情况一：删除后不影响平衡，直接删除

情况二：删除后不再是平衡二叉树



等价于21节点为RL型插入

先右旋后左旋解决

先右旋不平衡节点右孩子 再左旋不平衡节点

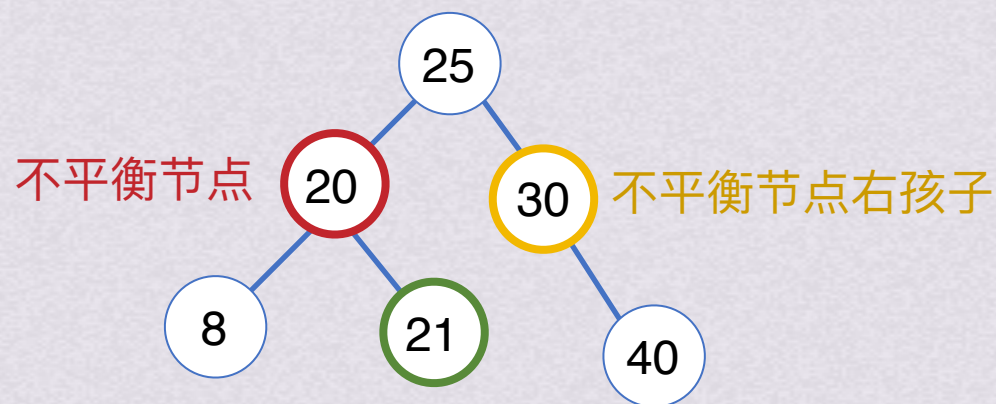


平衡二叉树删除操作

删除叶子节点情况

情况一：删除后不影响平衡，直接删除

情况二：删除后不再是平衡二叉树



等价于21节点为RL型插入

先右旋后左旋解决

先右旋不平衡节点右孩子 再左旋不平衡节点



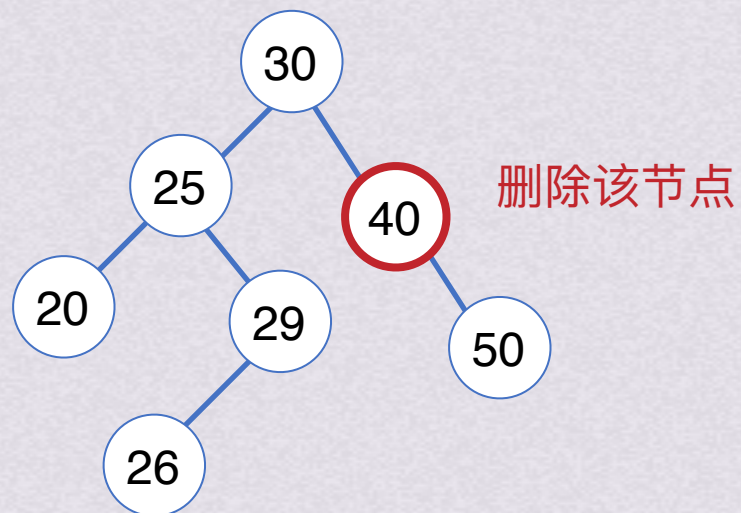
平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树



删除叶子节点情况

情况三：删除节点只有左/右孩子





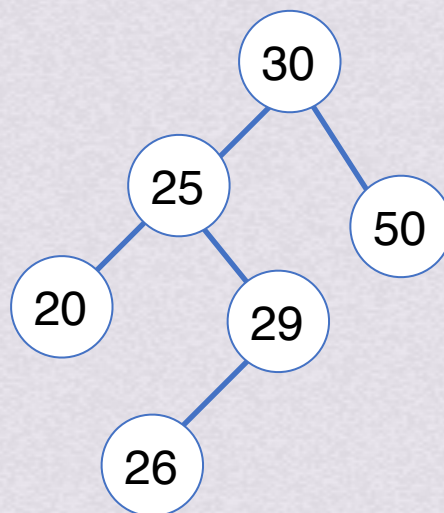
平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树



删除叶子节点情况

情况三：删除节点只有左/右孩子



删除该节点

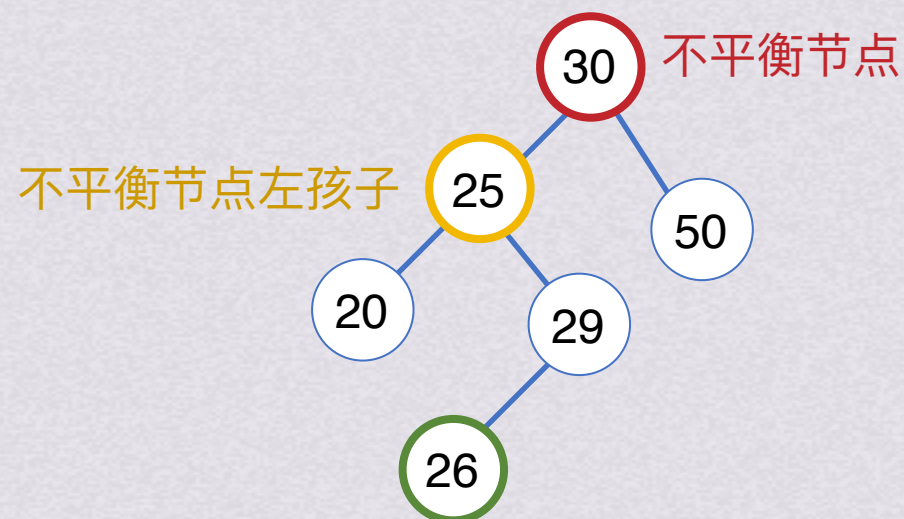


平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子



等价于插入了26节点
LR型插入，进行相应调整

先左旋不平衡节点左孩子 再右旋不平衡节点



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子



等价于插入了26节点
LR型插入，进行相应调整

先左旋不平衡节点左孩子 再右旋不平衡节点

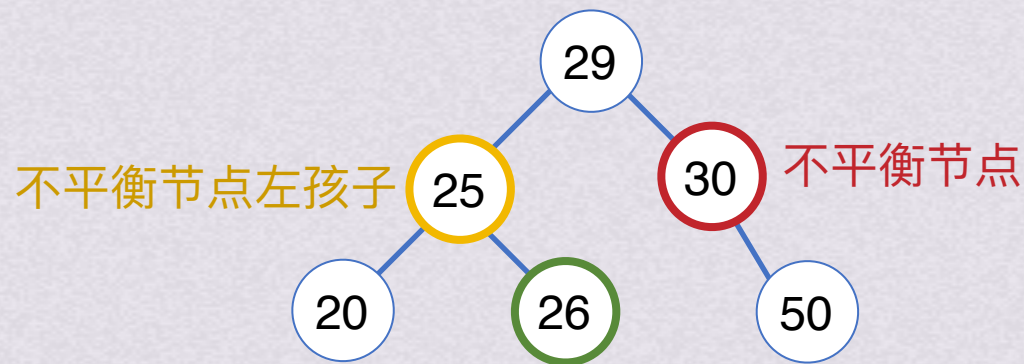


平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子



等价于插入了26节点
LR型插入，进行相应调整

先左旋不平衡节点左孩子 再右旋不平衡节点



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

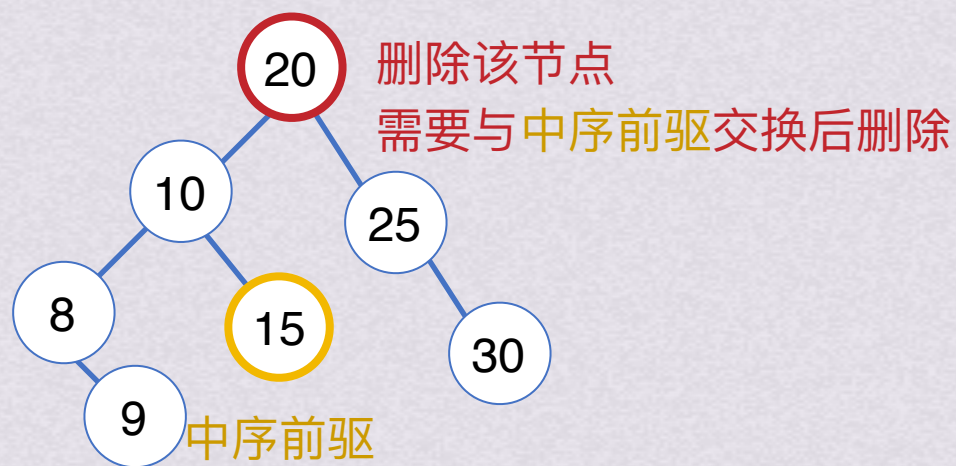


删除叶子节点情况

情况三：删除节点只有左/右孩子

删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1





平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

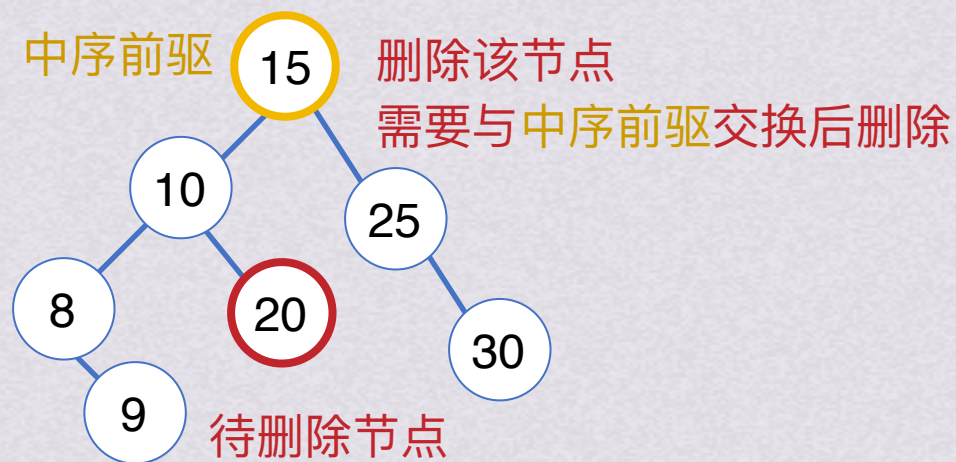


删除叶子节点情况

情况三：删除节点只有左/右孩子

删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1





平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子 删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1



等价于插入9，LR型插入

先左旋不平衡节点左孩子 再右旋不平衡节点



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树



删除叶子节点情况

情况三：删除节点只有左/右孩子 **删除节点同时有左右孩子的情况**

情况四：删除节点平衡因子为0/1



先左旋不平衡节点左孩子 再右旋不平衡节点



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子 删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1



先左旋不平衡节点左孩子 再右旋不平衡节点



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树



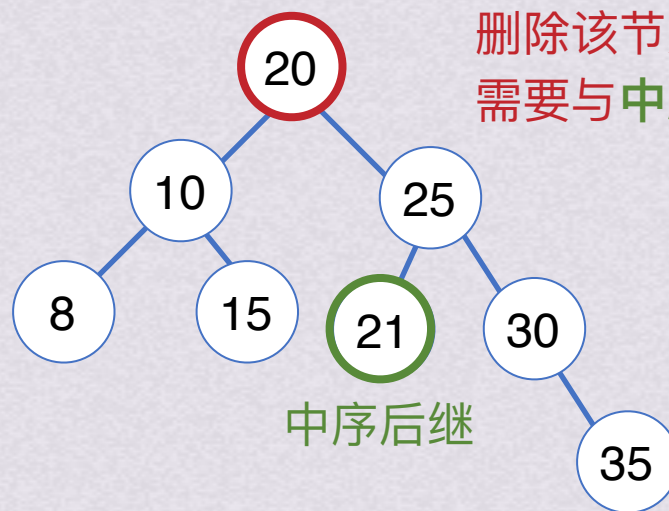
删除叶子节点情况

情况三：删除节点只有左/右孩子

删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1

情况五：删除节点平衡因子=-1



删除该节点

需要与中序后继交换后删除

中序后继



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

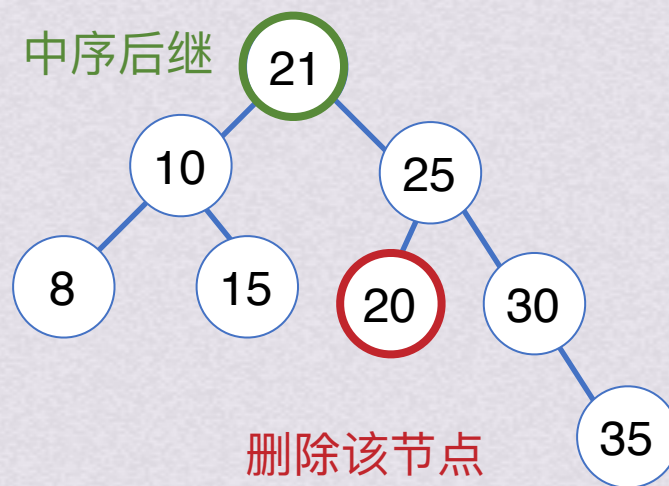


删除叶子节点情况

情况三：删除节点只有左/右孩子
情况四：删除节点平衡因子为0/1

删除节点同时有左右孩子的情况

情况五：删除节点平衡因子=-1



删除该节点
需要与中序后继交换后删除



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

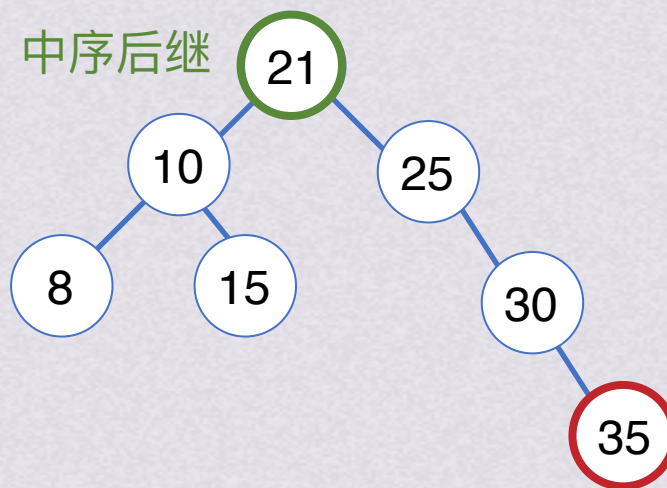
} 删除叶子节点情况

情况三：删除节点只有左/右孩子

删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1

情况五：删除节点平衡因子=-1



等价于插入35，RR型
左旋轻松解决



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树



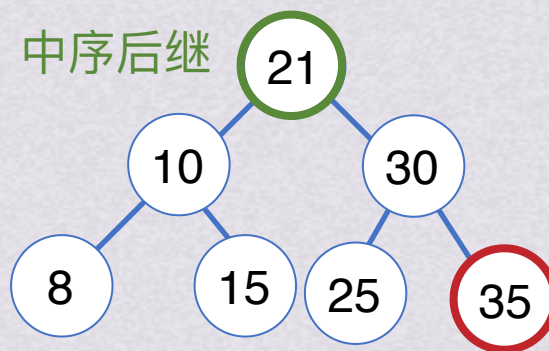
删除叶子节点情况

情况三：删除节点只有左/右孩子

删除节点同时有左右孩子的情况

情况四：删除节点平衡因子为0/1

情况五：删除节点平衡因子=-1



等价于插入35，RR型
左旋轻松解决



平衡二叉树删除操作

情况一：删除后不影响平衡，直接删除
情况二：删除后不再是平衡二叉树

} 删除叶子节点情况

情况三：删除节点只有左/右孩子

总体思路就是删除后转换为插入再调整

情况四：删除节点平衡因子为0/1

情况五：删除节点平衡因子=-1

} 删除节点同时有左右孩子的情况



平衡二叉树

创建操作

平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程



平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程

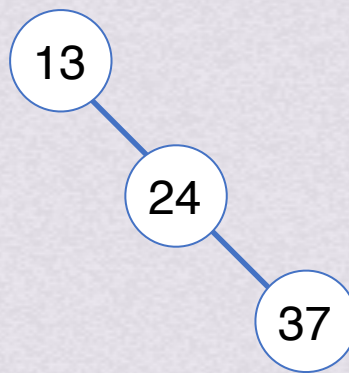
依次插入节点

13	24	37	90	53
----	----	----	----	----

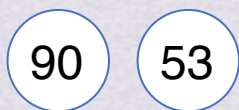


平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程



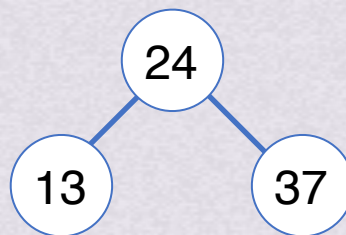
依次插入节点



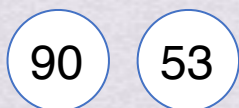


平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程



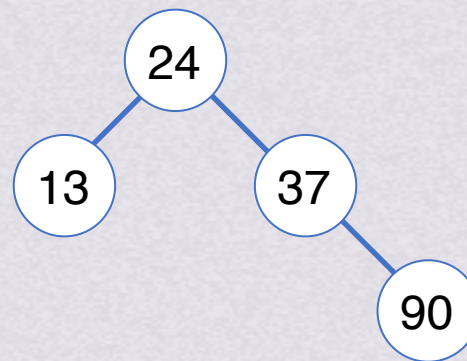
依次插入节点





平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程



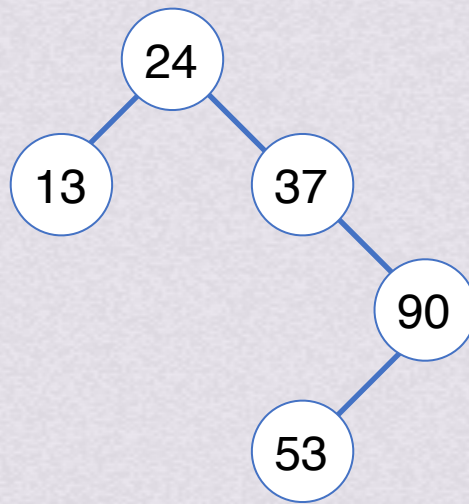
依次插入节点

53



平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程

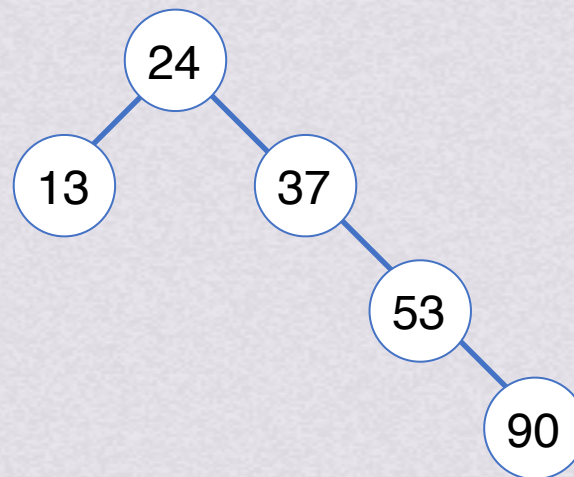


依次插入节点



平衡二叉树创建操作

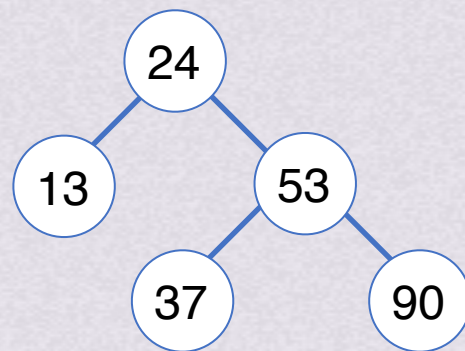
平衡二叉树的创建就是不断执行插入操作的过程





平衡二叉树创建操作

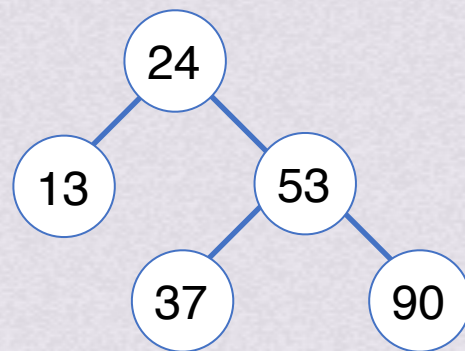
平衡二叉树的创建就是不断执行插入操作的过程





平衡二叉树创建操作

平衡二叉树的创建就是不断执行插入操作的过程



平衡树调整完毕



平衡二叉树

AVL

查找 插入 删除 创建